

**EC-390 Digital signal Processing
BS(CE)-2021**

**Digital Filter Design and Implementation
Project Report**



Submitted By:

Reg. No.	Name
21-CE-009	Huzaifa Shahbaz

Submitted To:

MAAM HAREEM KHAN

**Department of Computer Engineering
BS Computer Engineering Program
HITEC University Taxila**

Digital Filter Design and Implementation

Contents

Tables of figures.....	4
Chapter#1 Abstract.....	5
1.1 Abstract:	5
Chapter#2 Introduction.....	5
2.1 Introduction:.....	5
Chapter#3 Objective.....	5
3.1 Objective:.....	5
Chapter#4 Expected Outcome	5
4.1 Expected Outcome:	5
Chapter#5 Methodology	6
5.1 Methodology:.....	6
Chapter#6 Components Used	6
6.1 Components Used:	6
Chapter#7 Digital Filter Design	6
7.1 Digital Filter Design:.....	6
7.1.1 Finite Impulse Response:	7
7.1.2 Infinite Impulse Response:.....	7
Chapter#8 Functional Requirements of digital filter design.....	7
8.1 Functional Requirements:	7
8.1.1 Frequency Response Specifications:.....	7
8.1.2 Filter Type Selection:	7
8.1.3 Filter Order and Complexity:	7
8.1.4 Stability and Robustness:	8
8.1.5 Transition Bandwidth and Ripple:	8
8.1.6 Phase Linearity:	8
8.1.7 Implementation Constraints:	8
8.1.8 Filter Optimization:	8
8.1.9 Filtering Characteristics:.....	8
8.1.10 Validation and testing:	8

Chapter#9 Non-functional Requirements of digital filter design	8
9.1 Non-functional Requirements:.....	8
9.1.1 Performance:	8
9.1.2 Scalability:	9
9.1.3 Reliability:	9
9.1.4 Robustness:.....	9
9.1.5 Compatibility:	9
9.1.6 Maintainability:.....	9
9.1.7 Resource Efficiency:	9
9.1.8 Usability:.....	9
9.1.9 Portability:.....	9
9.1.10 Compliance:.....	9
Chapter#10 Code of digital Filter design	10
10.1 Code of Digital Filter Design:.....	10
Chapter#11 Kaiser Window	12
11.1 Kaiser Window:.....	12
Chapter#12 Results and Graphs.....	12
12.1 Results and Graphs:	12
12.1.1 Original Signal, Noisy Signal and Filtered Signal:	13
12.1.2 Filter Frequency Response:.....	13
Chapter#13 Conclusion	14
13.1 Conclusion	14

Tables of figures

Figure 1:Graph of original Signal, Noisy Signal and filtered Signal with respect to time	13
Figure 2:Graph of filter Frequency Response with respect to frequency and gain	13

Digital Filter Design and Implementation

Chapter#1 Abstract

1.1 Abstract:

In abstract, we discuss the digital filter design and implementation, distinguishing between finite impulse response (FIR) and infinite impulse response (IIR) filters, and highlighting key design methods like windowing and optimization algorithms. It highlights the use of software tools and their applications in audio, communications, and image processing.

Chapter#2 Introduction

2.1 Introduction:

Developing algorithms to process digital signals and eliminate undesirable elements or enhance desired aspects while maintaining signal characteristics is known as digital filter design. The impulse response of IIR filters is unlimited, whereas that of FIR filters is finite. Windowing, frequency sampling, optimization techniques, and software tools are all used in implementation. Image processing, telecommunication, and audio processing are examples of applications.

Chapter#3 Objective

3.1 Objective:

This technology improves signal quality, achieves desired frequency response, optimizes computational efficiency, ensures stability, facilitates parameter tuning, minimizes complexity, and integrates seamlessly into hardware and software platforms for practical deployment.

Chapter#4 Expected Outcome

4.1 Expected Outcome:

The goal of digital filter design and implementation is to use mathematical methods to obtain desired frequency response characteristics, such as low-pass, high-pass, band-pass, or band-stop. The expected result is a filter with the desired frequency domain features, such as attenuation or amplification, within specified passbands

and stopbands. Implementation often entails programming the filter algorithm in a digital signal processing (DSP) platform or hardware to ensure efficient calculation and real-time performance. The end product should show efficient suppression or isolation of specific frequencies while preserving others, thereby improving signal quality or extracting desired information. Testing and validation compare the filter's performance to design specifications, verifying that it is appropriate for its intended uses, such as audio processing, image enhancement, or communication systems. Continuous refining and optimization can be performed to improve filter efficiency and minimize computational complexity, or adapt to changing requirements.

Chapter#5 Methodology

5.1 Methodology:

To design a digital filter, define filter specifications, choose a filter type, design an analog prototype, apply frequency transformation, choose sampling frequency and discretization method, implement the digital filter, and verify performance through simulation and testing.

Chapter#6 Components Used

6.1 Components Used:

The components used of the digital filter design and implementation are following:

- 1.Visual Studio
- 2.Spyder

Chapter#7 Digital Filter Design

7.1 Digital Filter Design:

A digital filter uses a digital processor to perform numerical calculations on sampled values of the signal. The processor may be a general-purpose computer such as a PC, or a specialized DSP (Digital Signal Processor) chip. There are the following types of digital filter design are:

- 1.Finite Impulse Response
- 2.Infinite Impulse Response

7.1.1 Finite Impulse Response:

Finite Impulse Response (FIR) filters are a type of digital filter characterized by a finite duration of their impulse response, settling to zero in a finite number of steps. They exhibit linear phase response, making them suitable for applications requiring phase preservation. Implementation involves convolution of the input signal with the filter coefficients. The computational efficiency can be optimized using Fast Fourier Transform (FFT) techniques. Validation ensures the filter meets performance criteria, providing the expected frequency response and minimal distortion.

7.1.2 Infinite Impulse Response:

Infinite Impulse Response (IIR) filters are digital filters whose impulse response is theoretically infinite. They are created using analog filter design approaches such as Butterworth, Chebyshev, or elliptic filters, and then converted to the digital domain using bilinear transformation or matching z-transform. IIR filters use fewer coefficients than FIR filters to obtain comparable frequency response characteristics, making them more computationally efficient. However, feedback can cause instability and nonlinear phase response. Implementation entails recursive algorithms with outputs based on both current and previous input and output values. Proper design and validation are required to assure stability and compliance with specified criteria. Applications include audio processing, control systems, and communication.

Chapter#8 Functional Requirements of digital filter design

8.1 Functional Requirements:

The functional requirements of digital filter design are:

8.1.1 Frequency Response Specifications:

Define the filter's desired magnitude and phase response across several frequency bands.

8.1.2 Filter Type Selection:

Specify the type of filter (e.g., low-pass, high-pass, band-pass, or band-stop) based on the application requirements.

8.1.3 Filter Order and Complexity:

Determine the order of the filter, which influences its ability to meet the desired specifications while considering computational complexity.

8.1.4 Stability and Robustness:

Ensure that the filter design is stable under all conditions and robust against variations in input signals or parameters.

8.1.5 Transition Bandwidth and Ripple:

Ensure that the filter design is stable under all conditions and robust against variations in input signals or parameters.

8.1.6 Phase Linearity:

Specify the required phase response of the filter, especially in applications where phase distortion must be minimized, such as in audio or communication systems.

8.1.7 Implementation Constraints:

Consider constraints such as hardware limitations, processing speed, memory requirements, and real-time processing capabilities for practical implementation.

8.1.8 Filter Optimization:

Optimize the filter design to achieve the desired performance metrics while minimizing resources or meeting specific design constraints.

8.1.9 Filtering Characteristics:

Define any additional characteristics required, such as linear phase, group delay, or transient response, based on the application's needs.

8.1.10 Validation and testing:

Develop procedures to validate the designed filter against the specified requirements and ensure its effectiveness in real-world scenarios through rigorous testing and analysis.

Chapter#9 Non-functional Requirements of digital filter design

9.1 Non-functional Requirements:

The non-functional requirements of digital filter design are:

9.1.1 Performance:

Ensure the filter operates efficiently, with minimal processing delay, and meets real-time processing needs. This includes considerations for computational speed and resource usage.

9.1.2 Scalability:

Design the filter to handle varying signal sizes and sampling rates, allowing it to scale with the needs of different applications without significant redesign.

9.1.3 Reliability:

Guarantee the filter's consistent performance under different operating conditions, ensuring it does not introduce errors or fail unexpectedly during operation.

9.1.4 Robustness:

Ensure the filter can handle unexpected inputs or extreme conditions without degrading performance or causing system instability.

9.1.5 Compatibility:

Ensure the filter is compatible with existing hardware and software environments, facilitating easy integration into various systems and platforms.

9.1.6 Maintainability:

Design the filter to be easily modifiable and upgradable, allowing for adjustments or improvements without extensive rework.

9.1.7 Resource Efficiency:

Make sure the filter is optimized to utilize as little memory and processing power as possible to avoid unduly taxing the system into which it is integrated.

9.1.8 Usability:

Ensure that the filter design process and implementation are user-friendly, with clear documentation and straightforward configuration, enabling ease of use and deployment.

9.1.9 Portability:

Design the filter to be easily transferable across different systems or platforms, ensuring it can be deployed in diverse environments with minimal changes.

9.1.10 Compliance:

Adhere to industry standards and regulations, ensuring the filter design meets relevant compliance requirements for its intended application domain.

Chapter#10 Code of digital Filter design

10.1 Code of Digital Filter Design:

```
import numpy as np

import matplotlib.pyplot as plt

from scipy.signal import hamming, lfilter, freqz

np.random.seed(0) # For reproducibility

fs = 700 # Sampling Frequency

t = np.linspace(0, 1.0, fs) # Time vector

freq1 = 10 # Frequency of the first sine wave

freq2 = 80 # Frequency of the second sine wave

clean_signal = np.sin(2 * np.pi * freq1 * t) + np.sin(2 * np.pi * freq2 * t)

# Add noise

noise = np.random.normal(0, 0.5, clean_signal.shape)

noisy_signal = clean_signal + noise

# Step 2: Design the digital filter (low-pass FIR filter using window method)

cutoff = 10 # Desired cutoff frequency of the filter, Hz

nyquist = 0.5 * fs # Nyquist frequency

normal_cutoff = cutoff / nyquist # Normalized cutoff frequency

numtaps = 101 # Size of the FIR filter

# Use a Hamming window to design the filter

window = hamming(numtaps)

taps = np.sinc(2 * normal_cutoff * (np.arange(numtaps) - (numtaps - 1) / 2)) *
window

taps /= np.sum(taps) # Normalize the filter coefficients

# Step 3: Apply the filter to the noisy signal

filtered_signal = lfilter(taps, 1.0, noisy_signal)
```

```
# Step 4: Plot the results
plt.figure(figsize=(15, 10))

# Original signal
plt.subplot(3, 1, 1)
plt.plot(t, clean_signal)
plt.title('Original Signal')
plt.xlabel('Time [seconds]')
plt.ylabel('Amplitude')

# Noisy signal
plt.subplot(3, 1, 2)
plt.plot(t, noisy_signal)
plt.title('Noisy Signal')
plt.xlabel('Time [seconds]')
plt.ylabel('Amplitude')

# Filtered signal
plt.subplot(3, 1, 3)
plt.plot(t, filtered_signal)
plt.title('Filtered Signal')
plt.xlabel('Time [seconds]')
plt.ylabel('Amplitude')

plt.tight_layout()
plt.show()

# Plot frequency response of the filter
w, h = freqz(taps, worN=8000)
plt.figure()
```

```
plt.plot(0.5 * fs * w / np.pi, np.abs(h), 'b')
plt.title('Filter Frequency Response')
plt.xlabel('Frequency [Hz]')
plt.ylabel('Gain')
plt.grid()
plt.show()
```

Chapter#11 Kaiser Window

11.1 Kaiser Window:

It is a one-parameter family of window functions used in finite impulse response filter design and spectral analysis. The Kaiser window approximates the Diode Pumped Solid State (DPSS) window which maximizes the energy concentration in the main lobe but which is difficult to compute.

Chapter#12 Results and Graphs

12.1 Results and Graphs:

The results and graphs Of Digital filter design are shown following:

12.1.1 Original Signal, Noisy Signal and Filtered Signal:

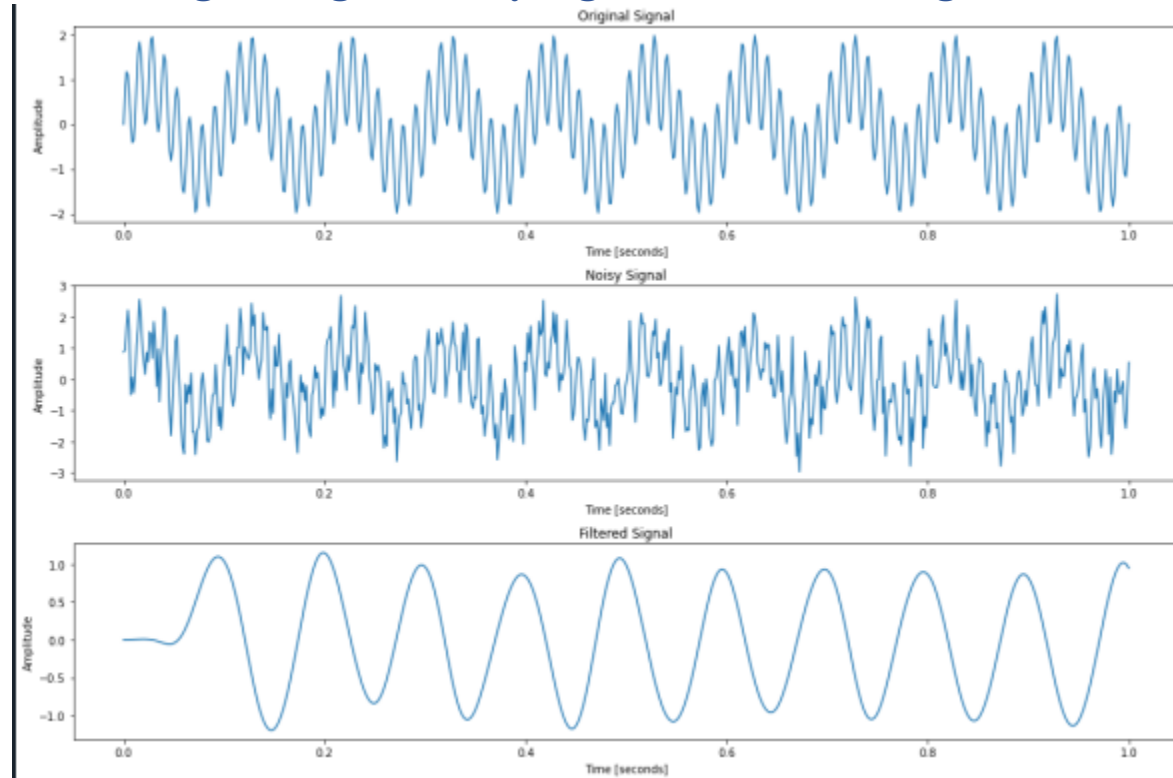


Figure 1: Graph of original Signal, Noisy Signal and filtered Signal with respect to time

12.1.2 Filter Frequency Response:

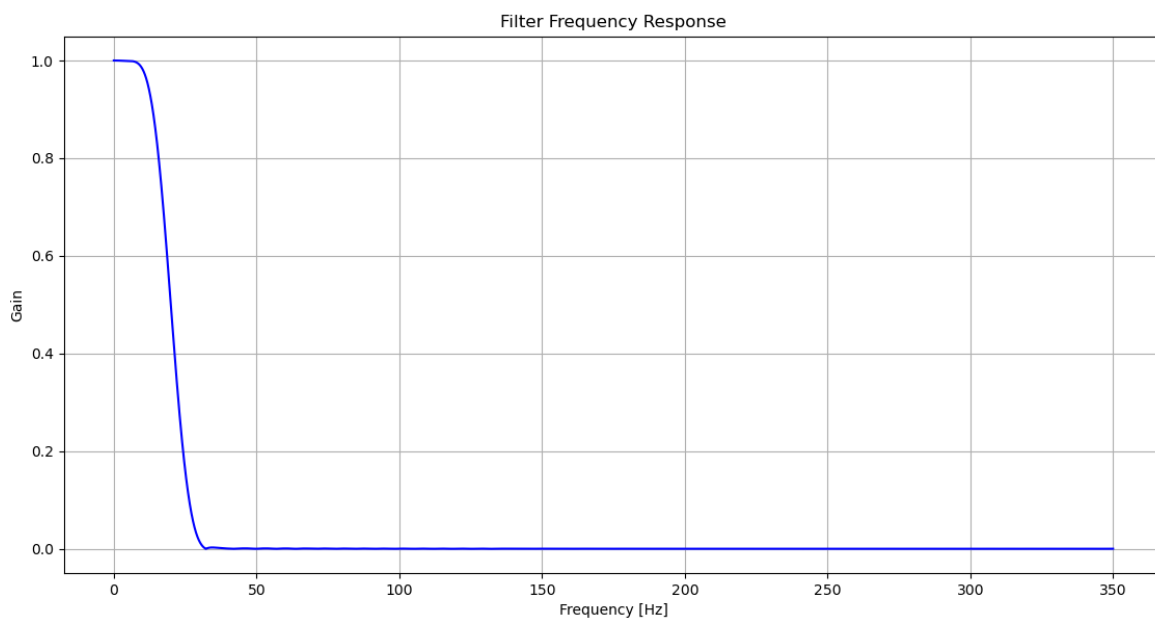


Figure 2: Graph of filter Frequency Response with respect to frequency and gain

Chapter#13 Conclusion

13.1 Conclusion:

In conclusion, this work gave a thorough approach to digital filter design, including both theoretical and practical aspects. The approach used to develop both Finite Impulse Response (FIR) and Infinite Impulse Response (IIR) filters was described, with a focus on their functional and non-functional needs. The Kaiser Window approach was recognized as an efficient way to improve filter performance. Several tests were conducted to confirm the implementation, demonstrating the filter's capacity to effectively reduce noise while keeping signal integrity. The findings and graphs gave a clear visual representation of the filter's performance, verifying the design's success.